

Featured Blog

Fast 2D shadows in Unity using 1D shadow mapping

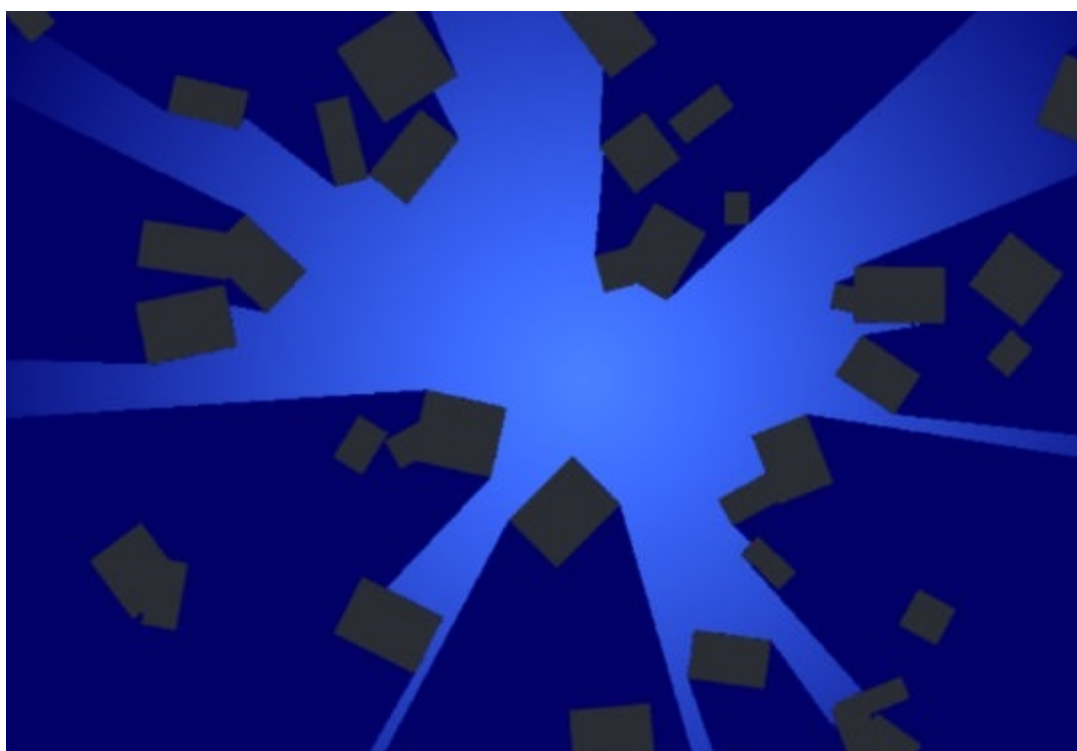
In this article I explain how I adapted a traditional technique from 3D rendering to achieve fast 2D shadows for a large number of light sources in the Unity game engine.

[Rob Ware](#)[Blogger](#)

February 26, 2018



Introduction



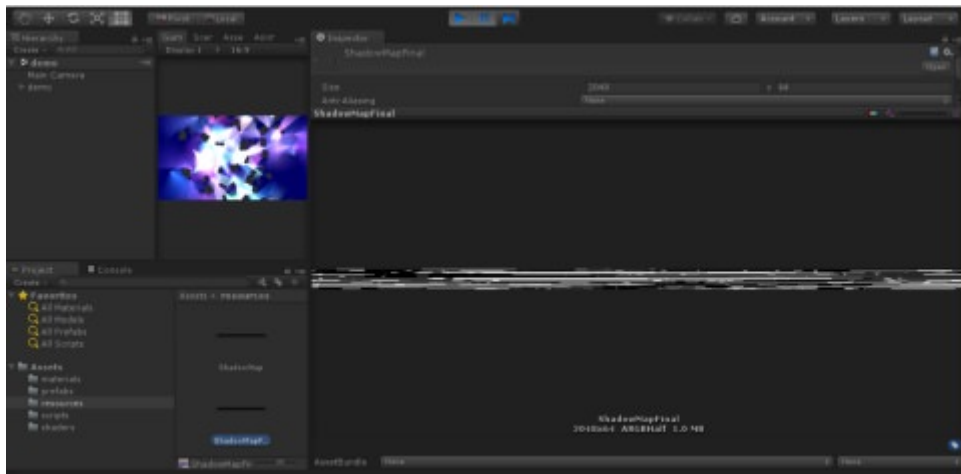
Recently I set out to create a 2D shadowing system for Unity which could be used in full game title. As all professional developers will know, there's a big difference between what can be achieved in a tech demo and what's suitable for integration into a full game in which the feature in question is just a small part of the experience. It needs to have a CPU, GPU and memory impact which plays nicely with everything else the game needs to do. In practical terms this varies between projects but I wanted something that didn't cost more than a couple of milliseconds of processing time, and not more than a few megabytes of memory.

This ruled out many of the pre-existing shadowing methods that I could find. There were a couple of techniques commonly in use. One would involve ray casting on the CPU to find the silhouette edges of light blocking geometry. Another would render all the light blockers into a texture and then run a multi-pass ray-stepping type algorithm over the texture to create a shadow map. These techniques are generally presented with at most a couple of light sources, they certainly wouldn't allow me to have dozens of shadow casting lights without exceeding the budget requirements that I have outlined.

Shadow Mapping

Instead I decided to create a 2D analogue of the way that most 3D games produce their shadows these days, that is, by rendering geometry from the point of view of the light source and creating a depth buffer which allows every pixel drawn later in rendering, to tell whether it is visible from the light source. This is the technique known as shadow mapping. In 3D this produces a 2 dimensional texture, whereas in 2D it produces a one dimensional texture. In the screenshot below you can see my resulting shadow map in Unity's resource view - but this is not for one light source, there are actually 64 - every row in the texture is a shadow map for a different light.

[SUBSCRIBE](#)[LOGIN/REGISTER](#)



This method relies on polar coordinates to convert a 2D position into an angle and a distance (or depth) relative to another object.

```
inline float2 CartesianToPolar(float2 point, float2 origin)
{
    float2 d = point - origin;

    // x is the angle in radians, y is the distance

    return float2(atan2(d.y, d.x), length(d));
}
```

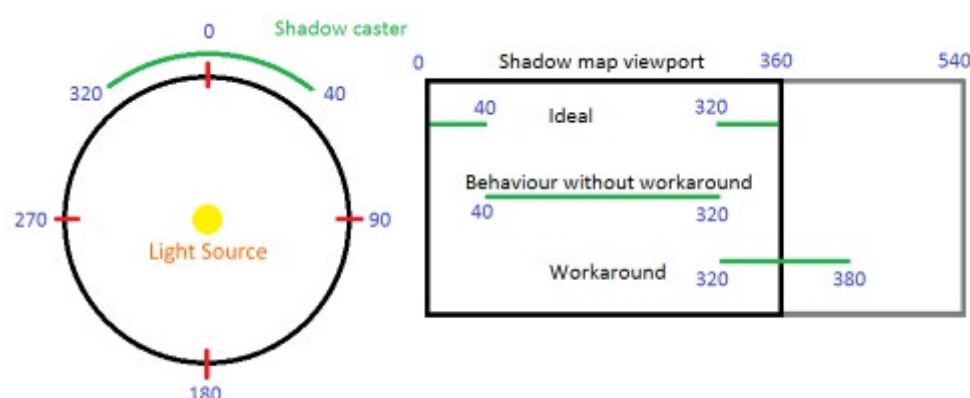
So each row in the shadow map represents the 360 degrees around the light source, and each pixel represents the distance from the light to the nearest solid geometry in that direction. The greater the horizontal resolution of the texture, the greater the fidelity of the resulting shadows.

The solid geometry which casts shadows, hereon termed *blocker geometry* is submitted as a line list. Each pair of vertices produce a pair of positions in polar space, and the pixel shader then fills in the corresponding segment in the shadow map, at each pixel only the closest pixel so far is stored, using the completely standard z-buffer test. It is not correct to simply interpolate the polar depth to produce the z coordinate in the pixel shader, this will produce curved shadows for straight edges. Instead you must compute the intersection point between and the line segment and the light ray at the current angle, but this is just a couple of dot products and a divide, so nothing too heavy for a modern GPU.

Complications

This whole thing would have been very simple were it not for one fly in the ointment – the big problem with the polar coordinate approach is when you get a line segment in polar coordinates which wraps around the 360 degree boundary. The usual approach would be to clip the line segment into two separate parts, the first part ends at 360 degrees, and the other part is the remainder of the segment, starting at 0. However a vertex shader takes one vertex and produces one result, there's no way to output two separate segments. The majority of the complexity in this solution is there to deal with this issue.

The solution is that actually the initial shadow map rows do not represent 360 degrees, they actually include an extra 180 degrees, that is from 0 to 540. A line segment is at most 180 degrees in polar space so this is enough to accommodate any segment which wraps around the 360 point. This means every line segment in still produces one line segment out, for the pixel shader, as it must.



The downside is that the first portion of the row (0 to 180), and the final portion (360 to 540), alias the same region in polar space. To test a pixel against the shadow map, you would have to determine whether the polar angle fell into this region, and if so, sample from two places and take the minimum of the two depths. This isn't something I wanted – the branching, and extra sampling, would be horrible performance-wise, especially when multi-sampling the texture for Percentage Closest

texture is so small by normal frame-buffer standards, this takes negligible GPU time. What we're left with is a final shadow map texture where a single sample is enough to tell whether a given pixel is lit by a given light.

The main drawback in this system is that the blocker geometry needs to be in a special format, in order to detect and deal with the wraparound case. Each vertex on each line segment is actually like a line segment in itself, because it stores the position of the other end of the line. This means you need to either build the geometry in this format at run time, or off line, you cannot just submit the geometry you use to render with, but on the plus side once you've built the special geometry, at least you're not submitting any unnecessary data and thus efficiency is greater.

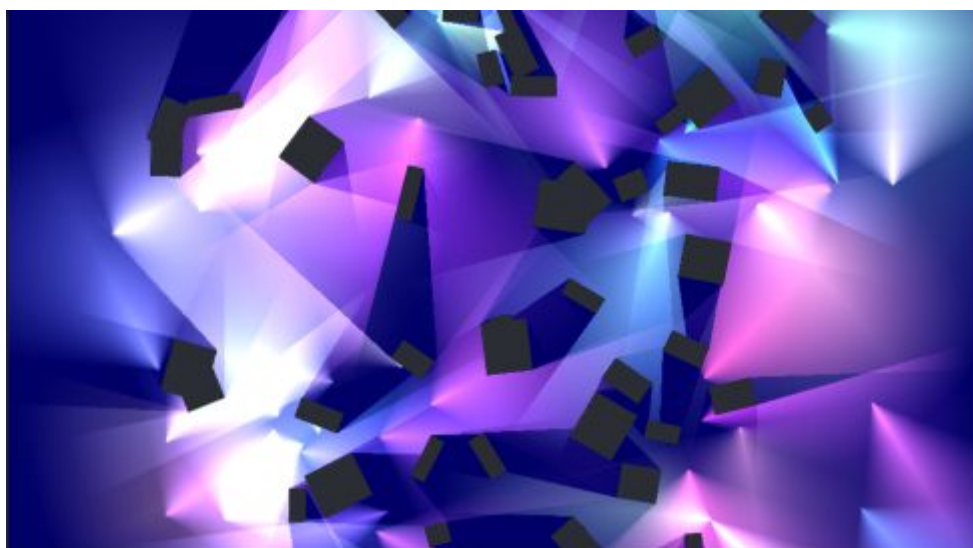
Another cool feature of this system, is that the final shadow map can be written back to the CPU and this makes it possible to do CPU based visibility queries without the need for ray casting. Copying the texture back to the GPU can be quite expensive, and although much awaited asynchronous gpu read-backs are heading towards us in Unity 2018, this isn't something you should do unless it's really needed.

Algorithm

In essence, this is how the algorithm works.

1. Create a mesh of line segments for shadow casting geometry. If the geometry doesn't change, there's no need to rebuild this every frame. You could also have two meshes, one for static geometry and another for dynamic geometry, to avoid unnecessary rebuilding.
2. Render this blocker geometry for each shadow casting light. In the demo it uses separate draw calls but this is a perfect opportunity to use instancing so that the mesh is only sent to the GPU once. Each light is assigned a row in the shadow map and this is passed to the shader through shader constants, so that they can produce a suitable y coordinate to write to.
3. In the vertex shader, line segments are transformed into polar space.
4. The gpu shades line segments in the shadow texture, on the current row, with a z test to keep the nearest pixels only.
5. Resample the final shadow map to another texture to eliminate the sections which alias the same polar region.
6. Render a quad for each light source covering the maximum possible range of the light (i.e. the radius for a point light), for each pixel, compute the polar coordinate with respect to the light source, and use this to sample the shadow map. If the polar distance is greater than the value read from the shadow map, this pixel is not lit by the light, so apply no lighting.

And here are the results. As a stress test, I set up 64 moving shadow casting point lights, with randomly sized light cones, moving between a number of rotating solid objects.



So what is the cost? Assuming your blocker geometry is static, and assuming that you use instancing, then the whole shadow map for any number of lights (subject to the limits for texture dimensions) can be sent to the GPU, in a single draw call. The amount of overdraw in the shadow map is determined by the complexity of your light blocking geometry, but since the texture is small compared to modern frame buffers, the GPU's cache performance should be fantastic. Similarly when you come to sample the shadow map, it's no different to 3D shadow mapping, except that the shadow map is much smaller.

Our game consists of a single large building environment, and we have 64 shadow casting lights active at all times, I use a shadow map texture of around 1024x64, and the costs are minimal within the overall frame budget.

Further possibilities



If you would like to take the system forward, there are a couple of interesting possibilities. When the shadow map is processed to remove the overlapping region, you could take the opportunity to convert the values and create an exponential shadow map, and then blur it (remember to only blur horizontally otherwise unrelated lights would bleed into each other!), which then allows you to do soft shadowing without multi-sampling the shadow map. Secondly, as mentioned earlier, the demo currently does a separate draw call for re-submit the shadowing geometry for each light, but if you pack the light position and other parameters into a matrix, it would be trivial to do this in a single draw call with instancing.

Furthermore, I believe it would be possible to do one-bounce radiosity lighting as an extension to this system, with almost no extra work on the CPU side. This is based upon the idea that the GPU can actually use the shadow map from the previous frame to work out where a light's rays bounce in the scene. I won't give away further details just now, because I haven't actually implemented it yet. If it works, it should be far more efficient than the usual Virtual Point Light implementations which relies on ray casting on the CPU.

It's also possible to use this system in other interesting ways. For instance, if you think of sound emitters instead of lights, then this system can be used to calculate audio occlusion. Or it could be used to determine line of sight to check for use in AI routines - in general, there's the potential to turn ray casts into texture lookups.

The end

This concludes the implementation details for my 1D shadow mapping solution. If you have any questions, please feel free to ask in the comments section below.

Demo source code:

<https://www.double11.com/misc/uploads/1DShadowMapDemo.zip>

[Console](#)

Get daily news, dev blogs, and stories from Game Developer straight to your inbox

Subscribe

LATEST JOBS

Infinity Ward

WOODLAND HILLS, CALIFORNIA

11.3.21

Disbelief

CAMBRIDGE, MASSACHUSETTS

11.3.21

SR. MULTIPLAYER DESIGN SCRIPTER/PROGRAMMER

JR. PROGRAMMER

4A Games

MALTA

11.3.21

XSEED

TORRANCE, CALIFORNIA

11.3.21

LEAD GAME SYSTEMS DESIGNER NEW IP

HEAD OF MARKETING

MORE JOBS



CONNECT WITH US

Register for a
Game Developer
Account

Gain full access to resources
(events, white paper, webinars,
reports, etc)
Single sign-on to all Informa
products

REGISTER

Subscribe to
Game Developer
Newsletter

Get daily Game Developer top
stories every morning straight into
your inbox

SUBSCRIBE

Follow us
@gamedevdotcom

Follow us @gamedevdotcom to
stay up-to-date with the latest
news & insider information about
events & more



Discover More From Informa Tech

Game Developers Conference

Game Developer Jobs

GDC Vault

Game Career Guide

Independent Games Festival

Game Developers Choice Awards

Omdia

Working With Us

Contact Us

About Us

Advertise



[Home](#) [Cookies](#) [Privacy](#) [Terms](#)

Copyright © 2021 Informa PLC Informa UK Limited is a company registered in England and Wales with company number 1072954 whose registered office is 5 Howick Place, London, SW1P 1WG.